# Lista Consolidada de Exercícios: Análise Léxica e Sintática

## Parte 1: Conjuntos FIRST e FOLLOW

### Exercício 1:

1. S -> A B C

2. A -> a A | ε

3. B -> b B | ε

4. C -> c C | d

5. D -> e

### Exercício 2:

1. E -> T E'

2. E' -> or T E' | ε

3. T -> F T'

4. T' -> and F T' | ε

5. F -> not F | id

### Exercício 3:

1. S -> i C t S E | a

2. E -> e S | ε

3. C -> b

4. X -> x

5. Y -> y

### Exercício 4:

1. S -> P S | K S | ε

2. P -> ( S )

3. K -> [ S ]

4. A -> S a

5. B -> b

## Exercício 5:

1. A -> V = E

2. V -> * V | id

3. E -> V | num

4. X -> E ;

5. S -> A | X

## Parte 2: Construção de Autômatos (AFD)

Instruções: Para cada padrão descrito abaixo, desenhe o Autômato Finito Determinístico (AFD) correspondente.

## Exercício 6: Números Reais (Float)

Construa um AFD que reconheça números reais. Regras: O número deve ter uma parte inteira (um ou mais dígitos), um ponto decimal obrigatório e uma parte fracionária (um ou mais dígitos). Ex: 10.5, 0.22.

## Exercício 7: Identificadores de Linguagem

Construa um AFD para identificadores. Regras: Devem começar obrigatoriamente com uma letra (A-Z, a-z). Após a primeira letra, podem conter qualquer combinação de letras ou dígitos (0-9). Ex: soma, valor1, x.

## Exercício 8: Comentários de Bloco (Estilo C)

Construa um AFD que reconheça comentários de bloco limitados por '/*' no início e '*/' no fim. O autômato deve aceitar qualquer sequência de caracteres entre os delimitadores.

## Exercício 9: Operadores de Comparação

Construa um único AFD que reconheça os seguintes tokens: '>', '>=', '<', '<=' e '==='. Dica: Use um estado inicial comum que ramifique para os diferentes operadores.

## Exercício 10: Strings Literais Simples

Construa um AFD que reconheça strings delimitadas por aspas duplas ("). Regras: A string começa com ", pode conter qualquer caractere exceto outra aspa dupla em seu interior, e termina obrigatoriamente com ". Ex: "texto".

## Parte 3: Reconhecimento de Cadeias e Tokens

Instruções: Analise o AFD descrito e determine a sequência de tokens gerada.

### Exercício 11

Descrição do AFD: Estado 0 (Inicial). Transições: [0-9] -> Estado 1 (Inteiro, Aceitação). No Est 1: [0-9] -> Estado 1.

Cadeia de Entrada: 123 45 6

Pergunta: Quais tokens são gerados?

### Exercício 12

Descrição do AFD: Estado 0 (Inicial). Transições: 'a' -> Estado 1 (Token_A, Aceitação); 'b' -> Estado 2 (Token_B, Aceitação). No Est 1: 'b' -> Estado 3 (Token_AB, Aceitação).

Cadeia de Entrada: ababa

Pergunta: Determine a sequência de tokens.

### Exercício 13

Descrição do AFD: Estado 0 (Inicial). Transições: '>' -> Estado 1 (OP_GT, Aceitação); No Est 1: '=' -> Estado 2 (OP_GE, Aceitação). Espaço ' ' reinicia o scanner.

Cadeia de Entrada: >= > >=

Pergunta: Liste os tokens na ordem de saída.

### Exercício 14

Descrição do AFD: Estado 0 (Inicial). Transições: [a-z] -> Estado 1 (ID, Aceitação). No Est 1: [a-z0-9] -> Estado 1. ':' -> Estado 2 (ATRIB, Aceitação sem valor). No Est 2: '=' -> Estado 3 (ATRIB_OP, Aceitação).

Cadeia de Entrada: x := y

Pergunta: Quais os tokens gerados (ignore espaços)?

### Exercício 15

Descrição do AFD: Estado 0 (Inicial). Transições: '/' -> Estado 1. No Est 1: '/' -> Estado 2 (COMENT_LINHA, Aceitação). '*' -> Estado 3. No Est 3: '*' -> Estado 4. No Est 4: '/' -> Estado 5 (COMENT_BLOCO, Aceitação).

Cadeia de Entrada: //teste /*fim*/

Pergunta: Identifique os tipos de comentários encontrados.